Flash: A Bounded Tool-Using Agent for Conversational Email and Calendar Management

Prof. Ann Mathews, Ronish Rohan, Nihal Rahman
Department of Computer Science and Engineering
CMR Institute of Technology, Bengaluru, India
ann.m@cmrit.ac.in, roro23cs@cmrit.ac.in, mdra23cs@cmrit.ac.in

Abstract—Gmail and Google Calendar already expose rich APIs, but actually using them still means clicking through search boxes, building queries by hand, and jumping between tabs. Flash is our attempt at removing that friction: a web assistant that turns a plain-language request into a bounded set of typed operations against both services. Under the hood it is a fairly ordinary stack - Next.js on the frontend, Supabase handling auth and row-level security, an OAuth 2.0 refresh cycle for Google tokens, and a DeepSeek-driven tool-use loop that streams its progress back to the browser as it works. The one design choice we insist on is that anything consequential - sending mail, or creating, editing, or deleting a calendar event - stops at a review card before it actually happens. In this paper we walk through the architecture, the tool protocol, the data model, and where the safety boundaries actually sit, and we back that up with an implementation-level verification: we read the source, inspected the database policies, and ran a production build rather than just describing the design on paper. What we found: 17 schemas are visible to the model, each agent run is capped at eight turns, all 13 API route modules require authentication, and row-level security is enabled on all four application tables. We want to be clear about what that does and doesn't show - it tells you the implementation is there, not that users finish tasks faster or with less effort. We also found real gaps worth naming: several lower-level mailbox mutations still execute without any confirmation step, content pulled from external email is not isolated from the model's instructions, tool call traces aren't persisted anywhere, and there is neither an automated test suite nor a user study behind any of this. So Flash isn't a finished product - it's a working baseline for thinking about human-approved, action-taking agents in this space.

Index Terms—large language model agents, email automation, Gmail API, Google Calendar API, human-in-the-loop, OAuth 2.0, tool use.

I. Introduction

Email has never really been just a communication channel - people use it as a to-do list, a filing cabinet, and a memory aid all at once. This isn't a new observation: early work already described inboxes as repositories for messages, tasks, and personal information, and tied heavier message volume to a felt sense of overload [1], [2]. Since then, mail clients have gotten better at reducing how much typing a reply takes, through things like Smart Reply and Smart Compose [3], [4]. But those tools help with one thing: producing text. They don't help when a request like "find the latest project thread, draft a response in my style, and check whether Friday afternoon is free" cuts across search, retrieval, composition, and scheduling all at once.

Large language models make it plausible to actually handle a request like that through conversation rather than four separate app interactions. ReAct showed that a model can interleave reasoning steps with real actions in an environment [5], and Toolformer showed that a model can learn on its own when an API call is worth making [6]. That's encouraging, but once the tools in question can modify someone's real mailbox or calendar, fluent text generation stops being the hard part. The harder problem is constraining what the model is allowed to touch, keeping identity consistent across the services it talks to, showing the user something meaningful while it works, and making sure anything with real consequences waits for a yes.

Flash is our attempt to work through that problem end to end, as a full-stack implementation rather than a proposal. A user signs in through Supabase, links a Google account via OAuth 2.0, and then just talks - the conversation persists across sessions. On the server side, a DeepSeek model gets access to a set of typed Gmail and Calendar tools, and whatever it produces streams back to the browser as text, status updates, or structured UI events. When the model wants to send a draft or touch the calendar, that call doesn't go straight through - it turns into a card the user has to approve, and approval routes through an authenticated application endpoint rather than letting the model finish the mutation itself.

What we're offering here is threefold, though we'd rather describe it plainly than list it: a layered architecture for how a model directs tool use without being handed full authority, a bounded execution-and-approval protocol that we can point to in the actual repository rather than just argue for, and a verification pass over that implementation that someone else could redo. We're deliberately not claiming Flash makes anyone faster, more accurate, or less burdened - we haven't measured that, so we don't say it.

A. Research Questions

Three questions shaped the work. First, how much mailbox and calendar capability can a productivity agent expose while still keeping its own decision process on a short leash? Second, where in the stack should approval

and identity checks actually live - in the model's loop, at the route boundary, or in the database? And third, once you have a working prototype like this, what's still missing before it could reasonably be called a deployable, safety-evaluated assistant?

II. Related Work

A. Email Assistance

The tension Whittaker and Sidner described - email being asked to do both communication and personal information management at once - is still the right frame for thinking about this problem [1]. Dabbish and Kraut's later finding, that message volume correlates with strain and that how people handle mail moderates that effect, points the same direction [2]: what people need help with is retrieval and action, not just writing faster.

Smart Reply took a neural sequence model and used it to generate short responses that make sense on a phone [3]. Smart Compose went further into real-time phrase completion, and its authors were candid about how much the latency budget of a production system shapes what's even possible [4]. Flash sits in a different part of the design space - the model here can search a mailbox, pull a thread, change labels, write drafts, and touch the calendar, not just suggest the next few words. That's a much bigger action space, and it drags in authorization and control questions that a text suggestion box never has to face.

B. Language Models as Tool Users

ReAct's contribution was really a pattern: let reasoning and acting interleave, and let the model use what it observes to steer its next step [5]. Toolformer took a related but distinct angle, showing a model can figure out on its own when an API call is worth the trouble [6]. We keep the operational core of both ideas - a loop of model calls, typed tools, and returned observations - but we don't leave the model unsupervised inside that loop. Schemas constrain what arguments look like, the loop itself has a hard turn limit, server-side code is what actually executes any service call, and a chosen subset of writes get bounced to the frontend for approval before anything happens.

C. Indirect Prompt Injection

Once a model can call tools, you inherit a confused-deputy problem: text pulled in from somewhere outside the conversation can carry instructions that compete with what the user actually asked for. Greshake et al. showed this concretely - applications that blend data and instructions together can be steered by content the model was never supposed to treat as commands [11], and InjecAgent later turned that into a benchmark for tool-using agents specifically [12]. Email is about as exposed to this as it gets, since anyone can put arbitrary prose into a message body that the assistant will eventually retrieve and read.

TABLE I
Threat-to-control mapping

Threat	Control	Residual gap
Cross-user data access	Route session checks and four RLS tables	Ownership boundaries are not integration-tested
Unintended send/event write	Email draft and three Calendar approval cards	No signed proposal or idempotency key
Runaway execution	Eight model iterations per chat request	Several tool calls can occur in one iteration
Token compromise	Server-side lookup, refresh, and RLS	Tokens are stored in database text fields
Injected content	Typed tools and selected approval gates	Retrieved text is not isolated from instructions

Human approval helps, but it's not a clean fix. People approve things that look plausible all the time without noticing they've been steered, and as it stands Flash still lets archive, trash, mark-read, label, and save-draft operations run without asking first. So approval is one layer among several here - it's not something we'd point to as proof the system is safe.

III. Design Goals and Threat Model

A. Design Goals

We settled on five things to optimize for. Boundedness: cap how many model-tool turns a single request can take. Identity continuity: every application request, conversation row, and OAuth token needs to trace back to one authenticated user, with no ambiguity. Reviewability: a confirmation step has to sit between the model deciding to do something high-impact and that thing actually happening. Transparency: the user should see named tool status and structured results as they stream in, not a spinner. Personalization: a writing persona, kept separate from everything else, should be able to shape how drafts get composed.

B. Assets and Failure Modes

What we're trying to protect: message content, recipients, calendar details, OAuth refresh tokens, saved conversations, and the user's writing persona. What an adversary can actually get their hands on: an email body, a subject line, a sender name, an event description, or in the worst case a direct prompt. The failures we care about follow from that - cross-user access, a leaked token, a mutation that happens twice or shouldn't have happened at all, the model following instructions it picked up from untrusted content, a tool loop that won't stop, and an interface that shows the user something misleading about what's actually going on.

IV. System Architecture

Flash's architecture separates the browser, authenticated Next.js routes, Supabase, and Google services (Table II). The dashboard sends chat requests to /api/chat; that route checks the Supabase session, loads the user's persona, runs the bounded agent loop, and streams

TABLE II
Architecture components and trust boundaries in Flash.

Component	Responsibilities	Boundary
Browser dashboard	Chat input, streamed text, data cards, voice input, and approval cards	Never calls Gmail or Calendar directly
Next.js server	Session/persona lookup, bounded 17-tool agent loop, NDJSON streaming, and authenticated write routes	Holds service credentials and enforces approval routes
Supabase	Authentication, OAuth tokens, conversations, messages, and persona	Row-level security isolates user data
Google services	OAuth refresh, Gmail API, and Calendar API	Accessed only through server-side wrappers

newline-delimited events back for text and cards. Approval cards submit reviewed email or Calendar payloads to separate authenticated write routes. Archive and trash are also routed through authenticated endpoints, but execute immediately when selected. The current dashboard additionally provides recoverable voice-input states, quick-start prompts, conversation browsing and deletion, and explicit loading, success, error, and retry feedback for Gmail actions.

A. Authentication and Account Connection

Supabase Auth handles who you are inside the app itself. Getting into someone's Google account is a separate step, done through the standard OAuth 2.0 authorization-code flow from RFC 6749 [7]. When the callback fires, we store the provider's refresh token, access token, expiry, and granted scope in gmail_tokens, keyed to the authenticated user. Before any service call goes out, the server checks whether the stored token still has more than 60 seconds of life left - if so, it reuses it; if not, it exchanges the refresh token and writes the new one back.

The prototype asks for Gmail read, modify, compose, and send permissions, plus Calendar access. Gmail models messages, threads, drafts, and labels as REST resources [8]; Calendar models calendars and events with their own organizer/attendee semantics [9]. It's worth being precise here: the OAuth scopes are an upper bound on what's possible, not what the model can actually do - the tool list we expose to it is narrower than that.

B. Conversation and Persona Persistence

Conversation metadata lives separately from the messages themselves. There's also an onboarding step where Flash can look at up to 40 of the user's own sent messages and ask a model to summarize the tone, structure, and phrasing it keeps repeating. That summary - the persona - gets stored once per user and tacked onto the system prompt whenever the model composes something. It's a cheap way to personalize output without any retraining, but it's honestly only as good as the sample of messages and the summary the model produces from them.

Algorithm 1: Flash request processing
1. Authenticate the user; load history and persona.
2. Resolve the Google token; expose tools if connected.
3. For turns 0 through 7, stream model events.
4. Forward text deltas to the browser.
5. If there is no tool use, terminate.
6. Append assistant tool calls to history.
7. Intercept UI, draft, and calendar-write calls.
8. Execute other server wrappers and append observations.

Fig. 1. Bounded request-processing loop used by Flash.

V. Bounded Agent Method

To make the loop concrete: let H_t be the accumulated dialogue at turn t, tool observations included, and let T be the set of 17 schemas. At each step the model either just answers in text or calls some $a_t \in T$. A server-executed call comes back with an observation o_t; an intercepted call comes back with an observation that just says a review card was shown to the user instead. That gives us

$$H_{t+1} = H_t \| a_t \| o_t, \qquad 0 \le t < 8. \tag{1}$$

The loop ends when the model stops asking for tools, produces nothing callable, hits an error, or simply runs out of turns. This bound keeps latency and repeated-action risk in check, but it says nothing about whether any given action was the right one.

Under the hood the chat route is just streaming newline-delimited JSON. Text events carry the prose as it's generated, tool events name whatever operation is currently running, and UI events carry a component identifier plus whatever structured data goes with it. That last piece matters more than it sounds - it means the browser can render email lists, message cards, event lists, drafts, and confirmation cards without ever asking the model to generate HTML directly.

VI. Tool, Data, and Approval Design

There are 17 schemas in the registry - eleven for Gmail, five for Calendar, and one for structured rendering. render_ui is frontend-only, while draft_email and the three Calendar write tools are intercepted as approval-card events before their corresponding authenticated route performs a service call. The remaining tools execute server-side wrappers directly. Typed schemas cut down on malformed calls, but they cannot tell whether the model chose the right target in the first place - that is a separate problem.

An approval-gated action happens in two phases. First, the model proposes - it fills in recipients, content, event identifiers, times, attendees, whatever the action needs. The server emits a UI event and tells the model that a draft or confirmation card was shown; no service call occurs at that point. Second, the user commits - if they approve, browser code sends the reviewed payload to an ordinary authenticated route, separate from the model entirely. The write route re-checks the session rather than

TABLE III
Declared tool catalog

Category	Count	Operations
Gmail retrieval	5	list, get, search, labels, thread
Gmail mutation/compose	6	draft, mark read, archive, trash, move, save draft
Calendar retrieval	2	events and calendars
Calendar write intent	3	create, update, delete
Presentation	1	render structured UI

TABLE IV
Persisted data and ownership

Relation	Ownership	Purpose
gmail_tokens	user equals authenticated subject	OAuth tokens and expiry
conversations	user equals authenticated subject	Chat metadata
messages	owner through conversation	User/assistant text
user_persona	user equals authenticated subject	Writing profile

trusting the proposal step. The current implementation does not attach a signed, persisted proposal identifier to that payload.

That boundary isn't symmetric, and we don't think it should be. Sending mail reaches outside the system, and editing an event can affect other people's calendars, so both require a look before they happen. Archive, trash, labeling, marking as read, and saving a draft all execute immediately, no review. That's a judgment call specific to this prototype, not a claim that those actions are universally low-risk - a production system would probably want confirmation to be configurable, based on the tool, the target, how many items are affected, and whether the action can be undone.

Four tables carry PostgreSQL row-level security: gmail_tokens, conversations, messages, and user_persona. The policies are straightforward - they check the authenticated subject against the row's owner, and for messages that ownership is checked through the parent conversation. This follows Supabase's own guidance: put row-level security on anything exposed, so authorization travels with the query instead of living somewhere it can be forgotten [10].

There's a list of safety properties we can't claim, and we'd rather say so plainly than gloss over it. OAuth tokens sit in plain text database fields, not application-level ciphertext. There's no durable audit log, no per-user rate limiter, no idempotency key anywhere, and nothing that explicitly marks an email body as untrusted data before it reaches the model. OWASP treats prompt injection and excessive agency as two separate risk categories [13], and Flash, at best, only partially addresses either one.

TABLE V
Verified implementation characteristics

Characteristic	Result	Interpretation
Model schemas	17	11 Gmail, 5 Calendar, 1 UI
Agent horizon	8 turns	Fixed request bound
Authenticated API modules	13 of 13	Session at route boundary
RLS app tables	4 of 4	Database user isolation
Approval intents	4	Send plus three Calendar writes
Models/effort	2 / 3	Runtime tradeoff
Production build	Passed cleanly	Next.js production compilation completed
Lint and type checking	Passed	Static checks completed without reported errors
Automated tests	None found	Behavior unmeasured

VII. Implementation Verification

A. Method

To be upfront: this is a repository-level verification, not a controlled experiment with participants. We went through the agent registry, the execution loop, the route handlers, the migrations, the OAuth refresh logic, the streaming client, and the onboarding code by hand, and separately ran the production build, linter, TypeScript checker, and repository diff checks. Every count below refers to one specific revision of the repository - it's evidence about the architecture, not a claim about what users experience.

We defined five checks going in: count the schemas and sort them by execution path; trace through what actually ends a run; enumerate every route and confirm it checks the session; match each application table against its row-level policy; and run the production compile plus static analysis. There was no test suite to run, no benchmark corpus, and no human-participant protocol - we're noting that rather than pretending otherwise.

B. Results and Interpretation

Flash mixes read and write access while still keeping the reasoning-action cycle on a leash - though it's worth flagging that a single turn can contain more than one tool call, so "eight turns" isn't the same thing as "at most eight requests to Gmail or Calendar." A tighter bound would need to count actual invocations, retries, and whatever they cost the service, not just turns.

All 13 API route modules we inspected check authentication, and row-level policies cover all four application tables - so you get defense in depth almost by construction: the handler establishes who's asking, and the database independently refuses to hand back rows that don't belong to them. Approval works the same way, split between the agent's stream and dedicated write routes rather than living in one place. Conversation creation now uses the server-issued identifier, and the first user message is

persisted against that conversation; conversation deletion also removes its associated messages. What's missing is any signed proposal identifier or idempotency control, so there's no durable record tying a specific review card to the commit that eventually followed it.

The production build, TypeScript check, and linter now complete cleanly. The Node-only Pi AI package is explicitly externalized through next.config.ts, removing the earlier dynamic module-resolution failure during bundling. Chat streaming also returns structured error events, and the client exposes recovery states for unsupported microphones and denied or failed microphone permissions rather than leaving the interface in an ambiguous listening state.

If there's one thing this design gets right, it's the seam between what the model decides and what the application is actually allowed to do. The model can only ask for operations that are registered, credentials live in server wrappers it never sees, and the interface - not the model - mediates whichever commits we chose to gate. Structured events also mean the browser never has to trust HTML the model generated. What this verification does not show: whether the model picks the right target, whether tone and style actually land, how time zones get handled, resistance to a genuinely malicious message, how long any of this takes, how much users trust it, or how it feels to use. Those need a real task corpus and people actually using it.

VIII. Limitations and Future Work

Retrieved email and event text lands in the model's context with no typed boundary separating it from instructions, which is exactly the opening indirect prompt injection needs [11], [12]. Confirmation covers sending mail and calendar writes, but not trash, archive, or bulk labeling - so there's a real gap there. Tokens depend on database access controls rather than encryption at the application layer. Nothing keeps an immutable audit trail, watches for anomalies, rate-limits per user, or runs automated tests. On top of that: tool calls have no idempotency keys, UI cards aren't saved alongside the conversation history, the schema only allows one Google connection per user, and attachments aren't modeled at all.

Any real evaluation going forward needs deterministic mailbox and calendar fixtures rather than a live account - each task should spell out which records it touches, which tools are allowed, which side effects are forbidden, and where the approval boundary sits. Worthwhile metrics include task success, how often the wrong target gets picked, unnecessary calls, approval precision, latency, cost, and how well the system recovers from its own errors. Adversarial test cases should specifically try to smuggle competing instructions into subject lines, bodies, signatures, quoted replies, and event descriptions - that's where this kind of attack actually shows up.

A production version of this would need to classify tools by how reversible they are, who they affect, and how much damage they could do; enforce real call and batch limits; add idempotency keys and let proposals expire; and log every proposal, approval, denial, and resulting service call to an actual audit table. Content coming in from the service should get parsed down into minimal typed fields rather than handed to the model as raw text that could pass for an instruction. Beyond that, there's a fairly long list of things worth building next: multiple Google accounts per user, attachments behind an explicit opt-in, persisted tool traces, proper time-zone handling, encrypted token storage, and adapters that aren't tied to one provider.

IX. Conclusion

What Flash shows, mainly, is that a conversational productivity agent can actually be put together out of ordinary parts - authenticated routes, OAuth-scoped service access, row-isolated persistence, typed tool schemas, streamed interface events, and human confirmation on the writes that matter most. In this revision that comes out to 17 exposed schemas, an eight-turn cap on the agent, authentication enforced across every inspected API module, and row-level security enabled on all four application tables.

We think the design holds up reasonably well specifically because the language model is never the final authority - something else always gets the last word. That said, we're not going to pretend it's finished: approval coverage has holes, untrusted content isn't isolated from instructions, there's no audit trail or idempotency mechanism, and none of the actual user-facing behavior has been measured yet. In the end, what makes an agent like this good isn't just how many tools it can call - it's how tightly its authority is bounded and how easily someone can go back and check what it actually did.

References

[1] S. Whittaker and C. Sidner, "Email overload: Exploring personal information management of email," in Proc. ACM CHI, 1996, pp. 276–283.

[2] L. A. Dabbish and R. E. Kraut, "Email overload at work: An analysis of factors associated with email strain," in Proc. ACM CSCW, 2006, pp. 431–440.

[3] A. Kannan et al., "Smart Reply: Automated response suggestion for email," in Proc. ACM SIGKDD, 2016, pp. 955–964.

[4] M. X. Chen et al., "Gmail Smart Compose: Real-time assisted writing," in Proc. ACM SIGKDD, 2019, pp. 2287–2295.

[5] S. Yao et al., "ReAct: Synergizing reasoning and acting in language models," in Proc. ICLR, 2023.

[6] T. Schick et al., "Toolformer: Language models can teach themselves to use tools," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[7] D. Hardt, "The OAuth 2.0 authorization framework," RFC 6749, IETF, Oct. 2012. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6749

[8] Google, "Gmail API overview," Google for Developers. [Online]. Available: https://developers.google.com/workspace/gmail/api/guides
[9] Google, "Google Calendar API concepts overview," Google for Developers. [Online]. Available: https://developers.google.com/workspace/calendar/api/concepts
[10] Supabase, "Row level security," Supabase Documentation. [Online]. Available: https://supabase.com/docs/guides/database/postgres/row-level-security
[11] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv:2302.12173, 2023.
[12] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," arXiv:2403.02691, 2024.
[13] OWASP Foundation, "OWASP Top 10 for large language model applications," 2025. [Online]. Available: OWASP LLM Application Security Top 10.